

5.3.2 IList<T> and IDictionary<K, V>

The immediate sub-interfaces of ICollection are IList and IDictionary which specify the behaviour for list-like and dictionary-like collections. Notably, list collections can be accessed via indexers such as arrays. Dictionaries can also be accessed by indexers, but instead of integers as in the case of lists, dictionaries are indexed by their key type (the K in the generic).

The two concrete implementations of lists and dictionaries that are provided by the .NET framework are List and Dictionary. List is implemented as an ArrayList, and Dictionary is implemented as a HashMap.

```
public static class ListDictTest
{
    public static void ListTest()
    {
        IList<int> list = new List<int>();
        list[0] = 5;
        list[3] = 8;
        list[25] = -2;
        foreach (int i in list) { Console.WriteLine(i); }
    }
    public static void DictionaryTest()
    {
        IDictionary<string, int> dict = new Dictionary<string,
int>();
        dict["John"] = 10;
        dict["Jane"] = 12;
        foreach (KeyValuePair<string, int> p in dict)
        {
            Console.WriteLine(p.Key + ", " + p.Value);
        }
    }
}
```

5.3.3 Stack, Queue, LinkedList, and others

In addition to lists and dictionaries, .NET provides more specialised collections. Stack and queue are obvious implementation of a stack and queue respectively. LinkedList and LinkedListNode are implementations of linked lists that deserves mention. LinkedLists are special in that they